

Inyeccion de argumentos (no es command injection)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ocurre cuando la app pasa tu entrada como ARGUMENTO a un binario que ya ejecuta (curl una URL, tar un fichero, ImageMagick una imagen), sin escaparla. No hay un shell que interprete ';': lo que abusas son las CAPACIDADES de esa herramienta. Con curl, por ejemplo, puedes colar una segunda URL o el esquema file:// para leer ficheros locales; con tar, flags como --checkpoint-action para ejecutar código.

El programador valida superficialmente 'la URL' o 'el fichero' pero construye la invocación concatenando tu texto en la línea de argumentos. La herramienta hace exactamente lo que le pides: curl acepta varias URLs y el esquema local file://, así que una entrada como 'http://ok file:///etc/passwd' se convierte en dos descargas.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                  -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando la salida delata que el backend envuelve una herramienta (aparece un medidor de progreso de curl, cabeceras, mensajes de wget/tar) y un ';id' NO ejecuta un segundo comando. Es la diferencia clave: si 'whoami' no corre pero un file:// o un flag si cambian el resultado, es argument injection, no command-injection.

Cómo identificarla

- La salida incluye trazas de una herramienta concreta (barra de progreso de curl, 'Resolving host...' de wget).
- Los separadores de shell no ejecutan comandos, pero cambiar la URL/fichero si altera el resultado.
- La función es de tipo 'importar/descargar/convertir desde una URL o fichero'.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Añade un segundo destino o un esquema local a la entrada (p.ej. una URL seguida de file:///etc/passwd) y observa si aparece contenido que la herramienta no debería traer.

Evidencia esperada: El contenido de un fichero local (raíz de /etc/passwd) o el efecto de un flag inyectado aparece en la respuesta, sin que exista un shell de por medio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`http://127.7 file:///etc/passwd`

Objetivo y contexto: curl acepta varias URLs en una invocación; el espacio separa un segundo argumento y `file://` es un esquema local. La primera URL supera la validación superficial y la segunda lee el fichero.

Resultado esperado: El contenido de `/etc/passwd` en la respuesta. Confirma inyección de argumentos/URL en curl (no command injection): la lectura ocurre por el esquema `file://`, no por un shell.

Anatomía de los payloads

La primera prueba es `http://127.7 file:///etc/passwd`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El contenido de un fichero local (raíz de `/etc/passwd`) o el efecto de un flag inyectado aparece en la respuesta, sin que exista un shell de por medio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica la herramienta por su salida (curl, wget, tar, convert, ffmpeg...).
- Descarta command injection: prueba `'id'` o `'&& id'`; si no ejecuta, no es un shell.
- Abusa de las capacidades del binario: con curl, una segunda URL y el esquema `file://` para leer ficheros; con tar/zip, flags que ejecutan hooks.
- Encadenalo con ssrf si primero hay que superar un filtro de URL para llegar a la invocación.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app usa una librería nativa (libcurl con parámetros separados, no la CLI) y no concatena una línea de argumentos.

- Un separador de shell (;, &&, |) SI ejecuta un segundo comando: entonces es command injection clasica, no inyeccion de argumentos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El backend ejecuta una herramienta (curl, wget, tar, convert) con tu entrada como argumento; no metes un comando nuevo, sino flags o una segunda URL/fichero que cambian lo que hace. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.

2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (argument-injection)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.